DATA STRUCTURE AND ALGORITHMS

LESSON 06
DOUBLE LINKED LIST

Conceptual Difficulty 

Implementation Difficulty 

LECTURER: SEAK LENG

OVERVIEW

1. Introduction to algorithm
2. Basic data types and statements
3. Control structures and Loop
4. Array
5. Data structure
6. Sub-programs

7. Recursive
8. Pointers
9. *Linked Lists*
10. Stacks and Queues
11. Sorting algorithms
12. Trees

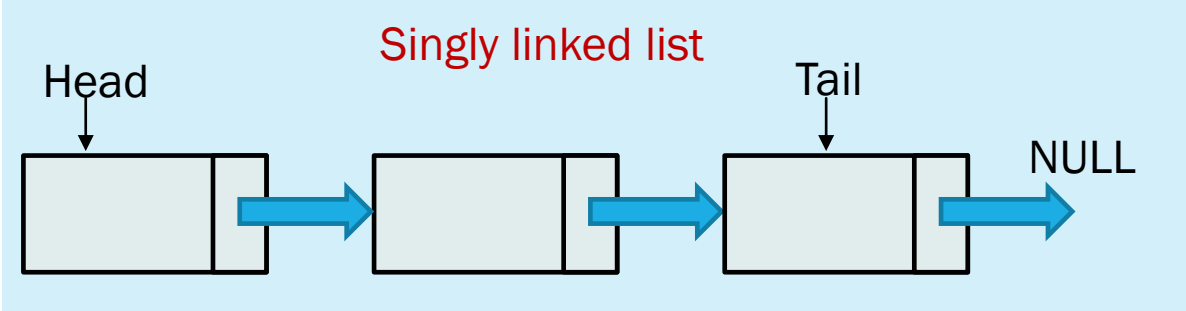
C++

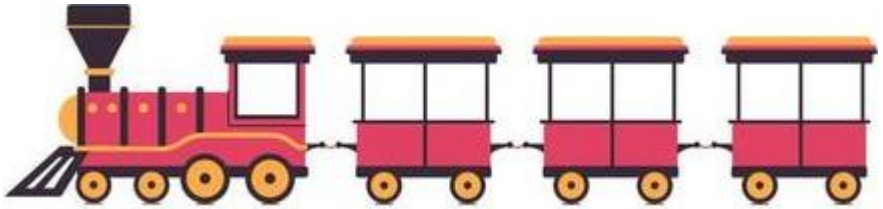
OUTLINE

- How to implement doubly linked list in C++
  - Create the list
  - Insert
- Examples

WHAT IS LINKED LIST?

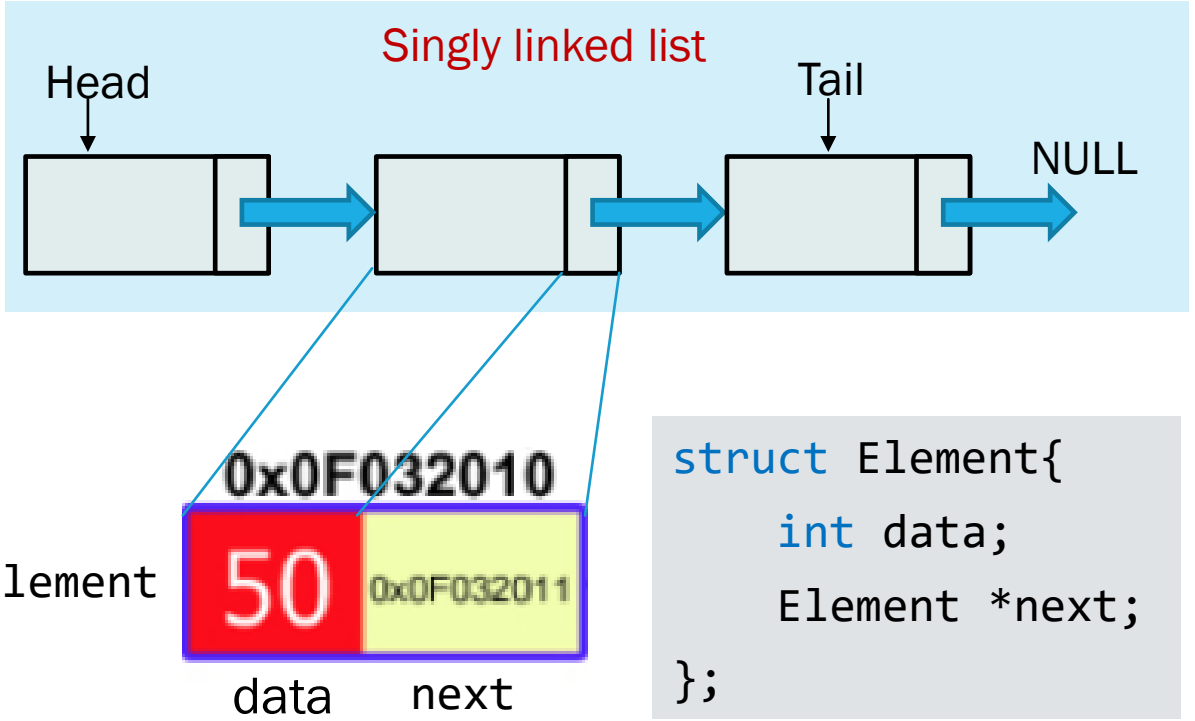
- A linked list is a linear data structure where elements are stored in nodes, and each node points to the next node in the sequence. Unlike arrays, linked lists don't require contiguous memory allocation.
- **Linked list** can store an indefinite number of elements/nodes (dynamic size).
- In a linked list, each element is linked with each other.





WHAT IS LINKED LIST?

- In a linked list, each element/node is linked with each other. Accessing to a linked list is done sequentially.
- Each element contains at least:
  - ✓ Data (can be any data type)
  - ✓ Ponter (link): a reference to its next element (**successor**) and/or previous element (**predecessor**)




```
struct Element{
    int data;
    Element *next;
};
```

C++ Procedural Code for Singly Linked List

WHAT IS LINKED LIST?

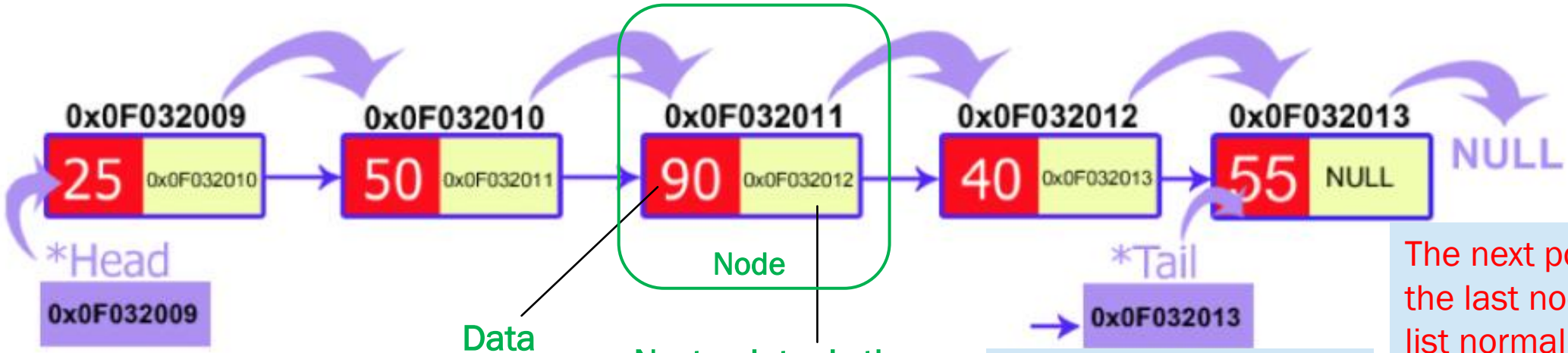
- ***Node:*** Contains data and a pointer to the next node
- ***Head:*** Pointer to the first node in the list, compulsory.
- ***Tail:*** Pointer to the last node in the list (optional, but useful)
- ***N:*** Number of nodes in the list (optional, but useful)

C++ Procedural Code for Singly Linked List:

```
struct Element{
    int data;
    Element *next;
};
```

```
struct List{
    int n;
    Element *head;
    Element *tail;
};
```

Example of Singly Linked List



Head is the pointer of the first node of the list

Tail is the pointer of the last node of the list

The next pointer of the last node of the list normally points to NULL

WHY USE A LINKED LIST?

Advantages:

- Easy to **add or remove** items (no need to shift everything like in arrays).
- **Flexible size** – it can grow or shrink easily.

Disadvantages:

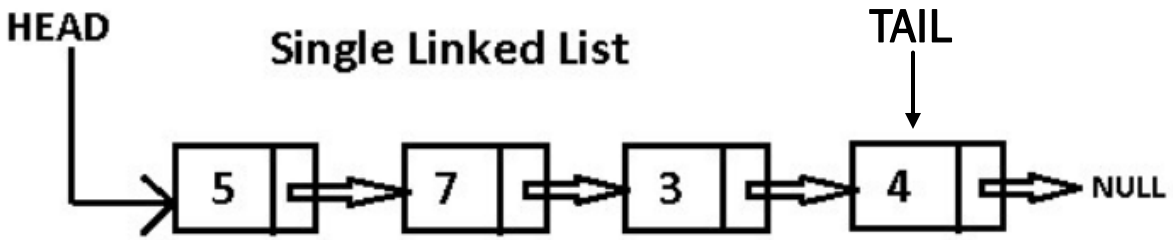
- **Slower access** – you have to go node by node to find something (no random access like arrays).

TYPES OF LINKED LIST

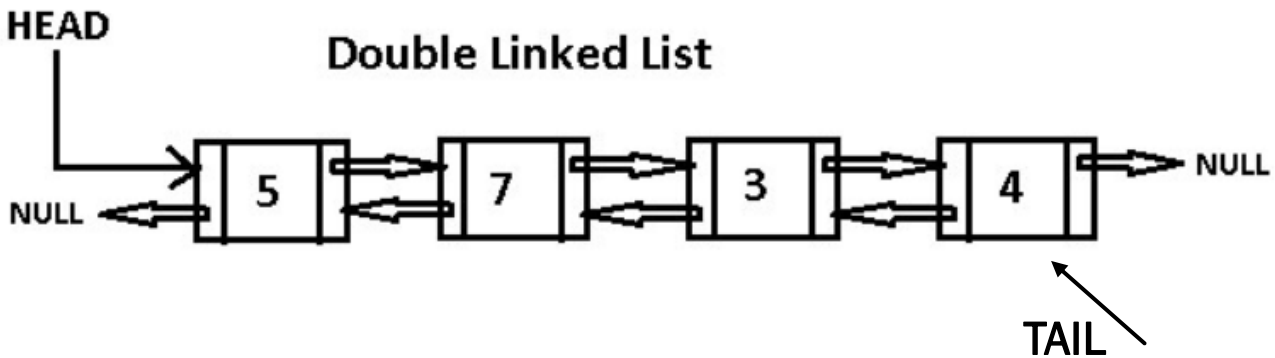
Successor : Next element
Predecessor : Previous element

- There are two main types of linked lists:
  1. A singly linked list is a linked list that has **a link to either its successor or predecessor**. Each node points only to the next node.
  2. A doubly linked list is a linked list that has **both links** to successor and predecessor.

```
struct List{
    int n;
    Element *head;
    Element *tail;
};
```




```
struct Element{
    int data;
    Element *next;
};
```



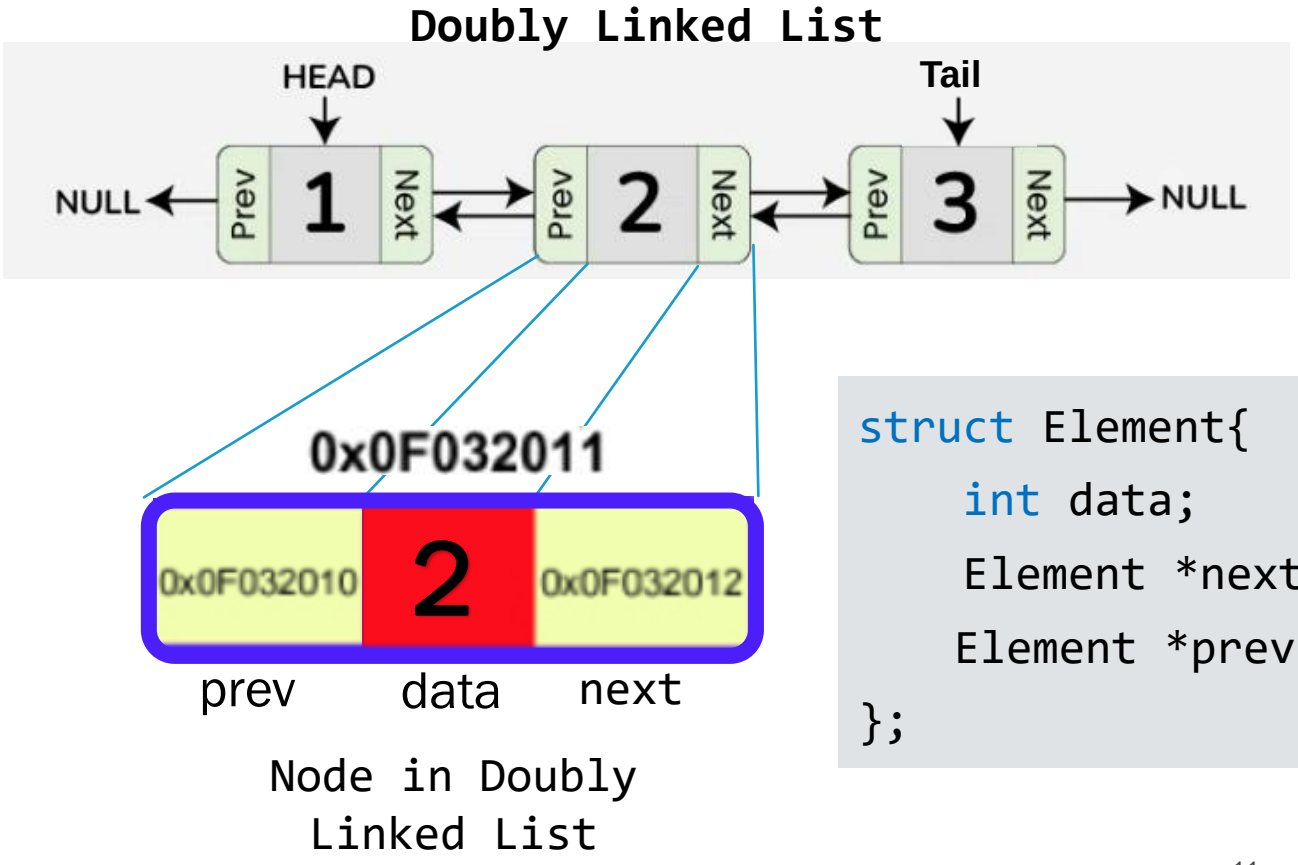

```
struct Element{
    int data;
    Element *next;
    Element *previous;
};
```

Doubly Linked List (SLL)

WHAT IS DOUBLY LINKED LIST?

Doubly linked list is a type of data structure that consists of a sequence of nodes, where each node contains 3 fields:

1. Data – the value stored in the node
2. Next – a pointer to the next node in the list
3. Prev – a pointer to the previous node in the list




```
struct Element{
    int data;
    Element *next;
    Element *prev;
};
```

DOUBLY LINKED LIST

- Advantages:
  - Bidirectional traversal (forward and backward)
  - Efficient deletion from the end, and display elements in reversing order
- Disadvantages:
  - Extra memory for previous pointer.
  - Slightly more complex implementation than singly linked list.

SINGLY LINKED LIST

- Create a linked list

1. Define a node structure: data, next, previous
2. Define a list structure: head, tail, n
3. Create an empty list

Steps to create an empty list:

1. Create a list variable
2. Allocate memory
3. Set 0 to n since we are creating an empty list
4. Head points to **NULL or nullptr**
5. Tail points to **NULL or nullptr**

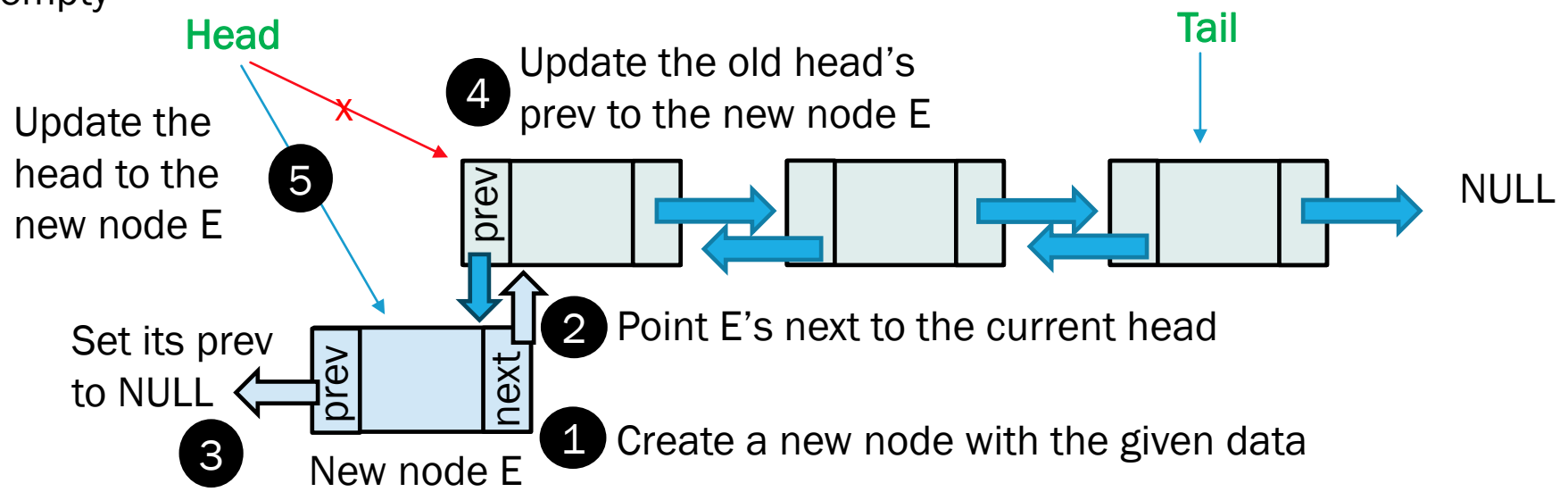
```
struct Element{
    int data;
    Element* next;
    Element* prev;
};

struct List{
    int n;
    Element* head;
    Element* tail;
};

List* createList(){

    List* ls;
    ls = new List();
    ls->n = 0 ;
    ls->head = nullptr;
    ls->tail = nullptr;
    return ls;
}
```

SINGLY LINKED LIST

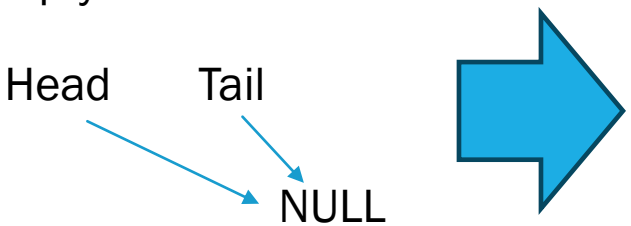
- Insert a new element to a list
  - Insert at the beginning
  - Insert at the end
  - Insert at a position

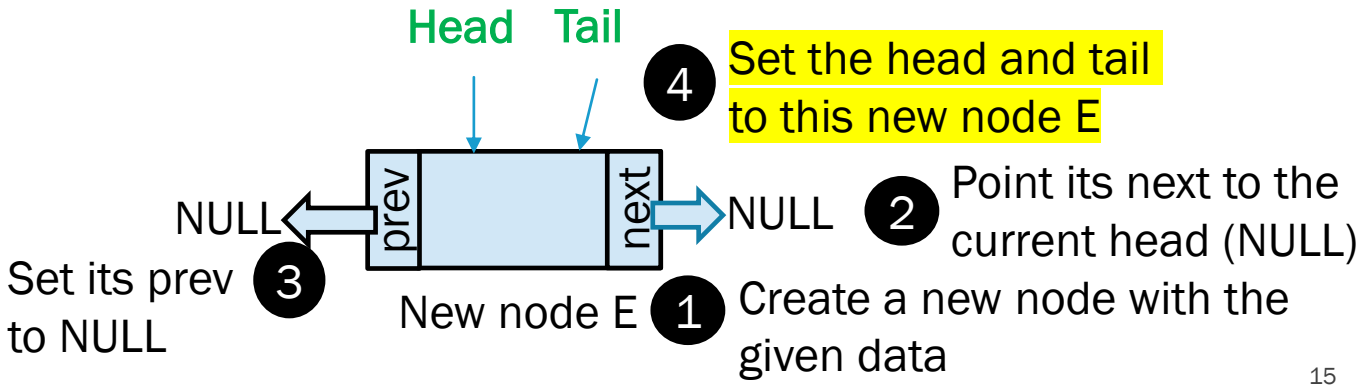
INSERTION AT THE BEGINNING OF THE LIST

The list is not empty



The list is empty





INSERTION AT THE BEGINNING OF THE LIST

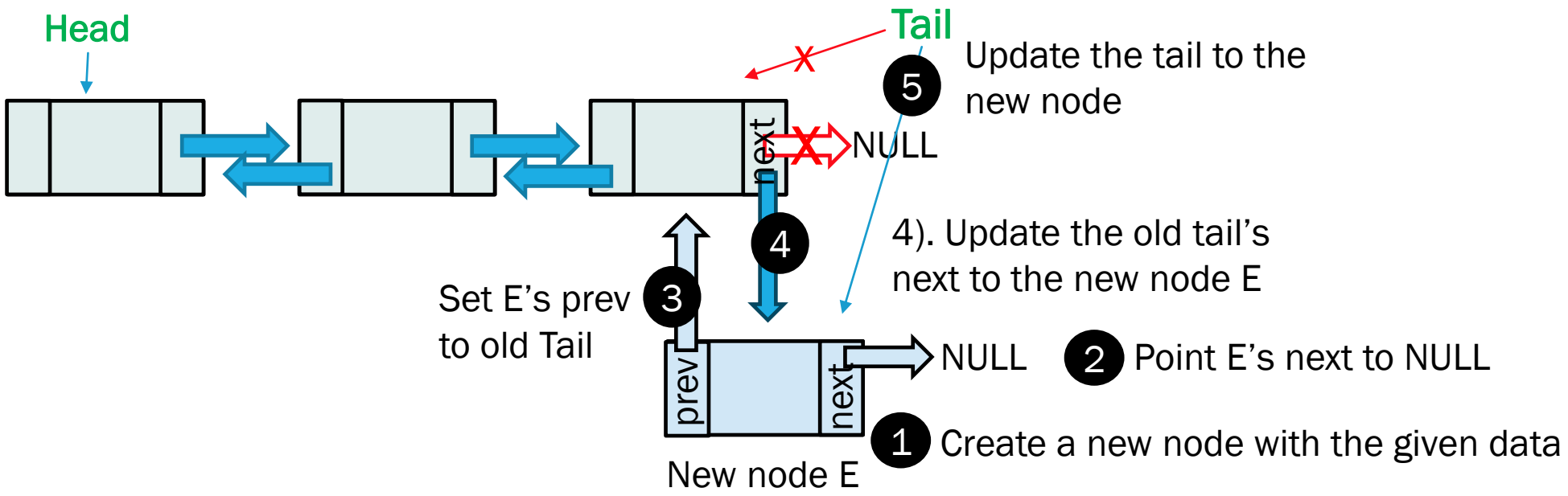
Steps to insert element at the beginning of a doubly linked list

1. Create a new element E
   - Allocate memory for a new node
   - Set its data to the given value
2. Link new node to the current head
   - Set next of new node to current head
   - Since it will become the first node, previous is null pointer.
3. If the list is empty
   - Both head and tail must point to the new node
4. If the list is not empty
   - Set the current head's previous pointer to the new node
5. Update the list head
   - Make the new node the new head
6. increment size of the list
   - Increase the number of elements in the list

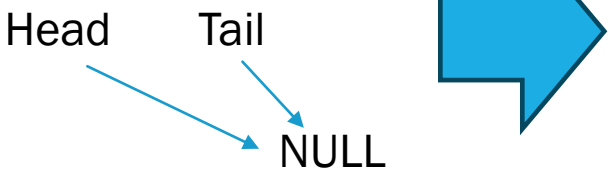
```
void addBeg(List* ls, int data){
    Element* e = new Element;
    e->data = data;
    e->next = ls->head;
    e->prev = nullptr;

    if(ls->n==0){
        ls->tail = e;
    }
    else{
        ls->head->prev = e;
    }
    ls->head = e;
    ls->n++;
}
```

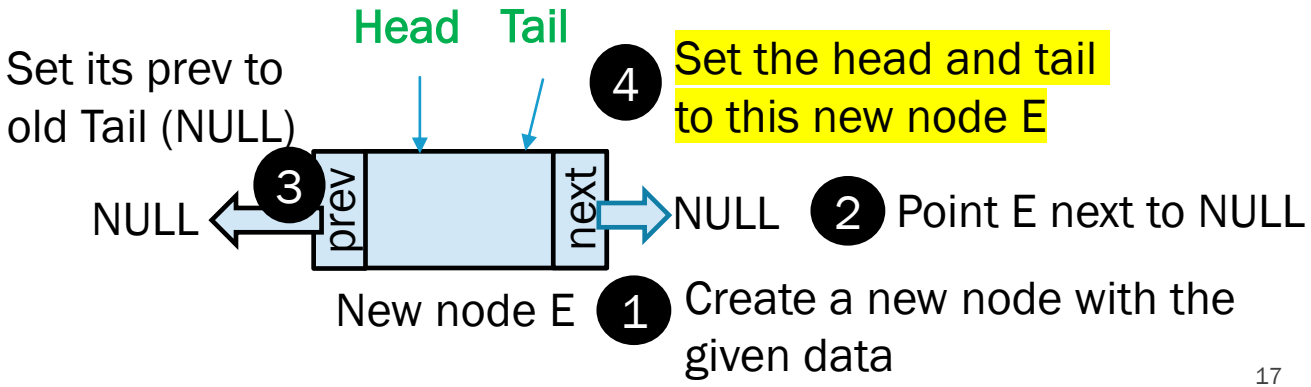
INSERTION AT THE END OF THE LIST

The list is not empty



The list is empty





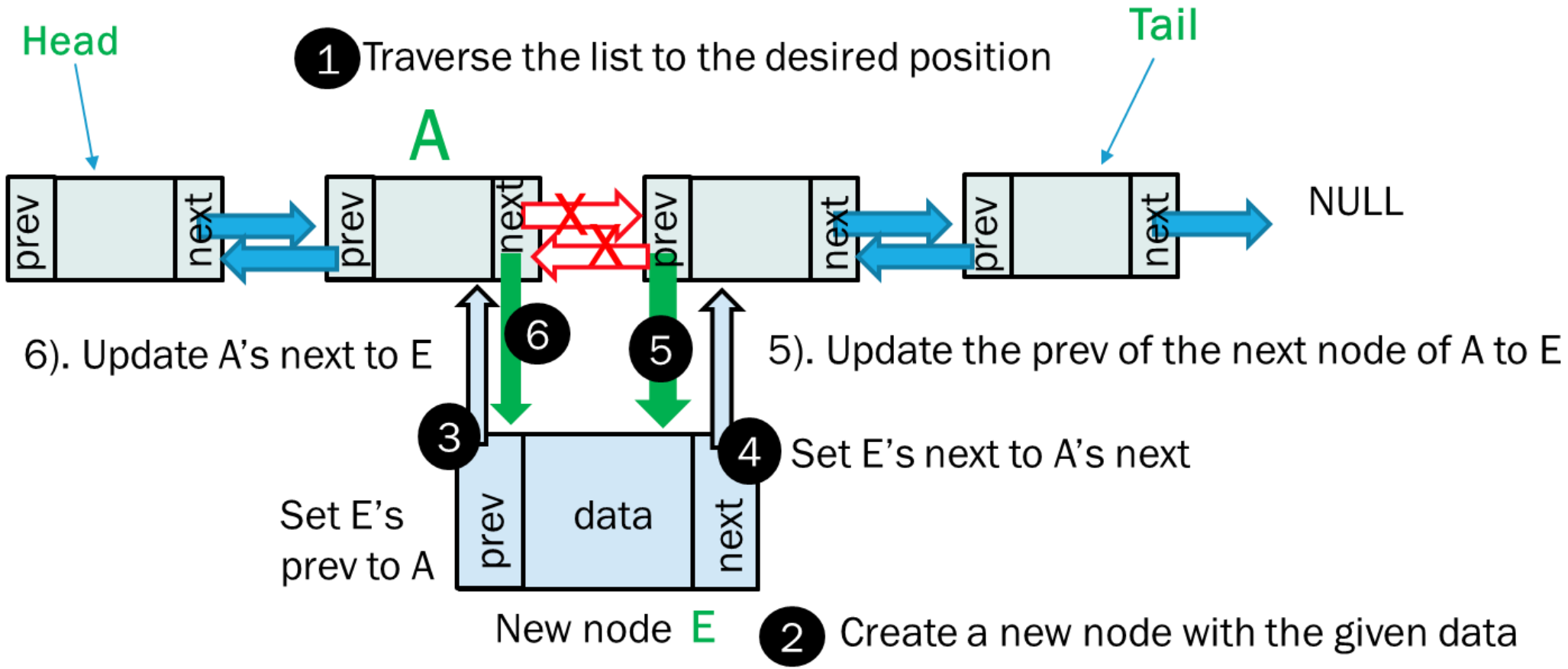
INSERTION AT THE END OF THE LIST

Steps to insert element at the end of a doubly linked list

1. Create a new element E
   - Allocate memory for a new node
   - Set its data to the given value
   - Set the next pointer to null since it is the last element
   - Set the previous pointer to the current tail to link back to current tail if exists
2. If the list is empty
   - Both head and tail must point to the new node
3. If the list is not empty
   - Set the current tail's next pointer to the new node
4. Update the list's tail to point to the new element
   - Make the new node the new tail
5. increment size of the list
   - Increase the number of elements in the list

```
void addEnd(List* ls, int data){
    Element* e = new Element;
    e->data = data;
    e->next = nullptr;
    e->prev = ls->tail;
    if(ls->n==0){
        ls->head = e;
    }else{
        ls->tail->next = e;
    }
    ls->tail = e;
    ls->n++;
}
```

INSERT THE ELEMENT AT POSITION



INSERTING AN ELEMENT AT POSITION

Steps to insert element at the position of a doubly linked list

1. If the position is invalid, exit
2. If inserting at position 0, call add_begin
3. If inserting at end, call add_end
4. Else:
   - Traverse the list to the desired position
   - Insert the new node before the current node:
     - Update the previous and next pointers of the surrounding nodes.
     - Update the list size n

```
void addPos(List* ls, int data, int pos){
    if(pos <0 || pos > ls->n){
        return;
    }
    if(pos==0){
        addBeg(ls, data); return;
    }
    if(pos==ls->n){
        addEnd(ls, data); return;
    }
    Element* current = ls->head;
    for(int i=0;i<pos-1;i++){
        current = current->next;
    }

    Element* e = new Element;
    e->data = data;
    e->prev = current;
    e->next = current->next;

    current->next->prev = e;
    current->next = e;
    ls->n++;
}
```

TRAVERSAL

A. Forward

Steps to show element from first to last nodes in a doubly linked list

1. Start at the first node (head)
2. While the current node is not empty
   - Print the data
   - Move to the next node

```
void displayBeg(List* ls){
    Element* e = ls->head;
    while(e!=nullptr){
        cout << e->data << " ";
        e = e->next;
    }
    cout << endl;
}
```

TRAVERSAL

B. Backward

Steps to show element from last to first nodes in a doubly linked list

1. Start at the last node (tail)
2. While the current node is not empty
   - Print the data
   - Move to the previous node

```
void displayEnd(List* ls){
     if(ls->n==0) return;
     cout << "List in reverse: " << endl;
     Element* e = ls->tail;
     while(e!=nullptr){
          cout << e->data << " ";
          e = e->prev;
     }
     cout << endl;
}
```

IMPLEMENTATION OF DOUBLY LINKED LIST IN C++

```cpp
#include <iostream>
using namespace std;

struct Element{
    int data;
    Element* next;
    Element* prev;
};

struct List{
    int n;
    Element* head;
    Element* tail;
};

List* createEmptyList(){
    List* ls = new List;
    ls->head = NULL;
    ls->tail = NULL;
    ls->n =0;
    return ls;
}
```

```cpp
List* createEmptyList(){
    List* ls = new List;
    ls->head = NULL;
    ls->tail = NULL;
    ls->n =0;
    return ls;
}

void addBeg(List* ls, int data){
    Element* e = new Element;
    e->data = data;
    e->next = ls->head;
    e->prev = nullptr;

    if(ls->n==0){
        ls->tail = e;
    }
    else{
        ls->head->prev = e;
    }
    ls->head = e;
    ls->n++;
}
```

```cpp
void addEnd(List* ls, int data){
    Element* e = new Element;
    e->data = data;
    e->next = nullptr;
    e->prev = ls->tail;
    if(ls->n==0){
        ls->head = e;
    }else{
        ls->tail->next = e;
    }
    ls->tail = e;
    ls->n++;
}
```

IMPLEMENTATION OF DOUBLY LINKED LIST IN C++

```
void addPos(List* ls, int data, int pos){
    if(pos <0 || pos > ls->n){
        return;
    }
    if(pos==0){
        addBeg(ls, data); return;
    }
    if(pos==ls->n){
        addEnd(ls, data); return;
    }
    Element* current = ls->head;
    for(int i=0;i<pos-1;i++){
        current = current->next;
    }

    Element* e = new Element;
    e->data = data;
    e->prev = current;
    e->next = current->next;

    current->next->prev = e;
    current->next = e;
    ls->n++;
}
```

```
void displayBeg(List* ls){
    cout << "List has " << ls->n << " elements: " << endl;
    Element* e = ls->head;
    while(e!=nullptr){
        cout << e->data << " ";
        e = e->next;
    }
    cout << endl;
}
```

```
void displayEnd(List* ls){
    if(ls->n==0) return;
    cout << "List in reverse: " << endl;
    Element* e = ls->tail;
    while(e!=nullptr){
        cout << e->data << " ";
        e = e->prev;
    }
    cout << endl;
}
```

```
int main(){
    List* list1 = createEmptyList();
    for(int i=0;i<10;i++){
        addBeg(list1,i);
    }
    displayBeg(list1);

    List* list2 = createEmptyList();
    for(int i=10; i<20;i++){
        addEnd(list2, i);
    }
    displayEnd(list2);

    addPos(list2, 99, 3);
    displayEnd(list2);
}
```

DOUBLY LINKED LIST

- Delete an element from a list
  - Delete at the beginning
  - Delete at the end
  - Delete at the position

How to delete data from linked list

DELETE FROM THE BEGINNING OF DLL

Steps to delete a node from a specific position in a doubly linked list (DLL)

1. If the list is empty, do nothing and return
2. Save the first node (head) in a temporary variable
3. Move the head to the next node
4. If the new head is not null, set its prev to null
5. Else, the list is empty , so set tail to null
6. Delete the old head (free memory)
7. Reduce the count of nodes in the list by 1

```
void deleteBeg(List* ls){
    if(ls->n==0) { return; }

    Element* temp = ls->head;
    ls->head = temp->next;
    if(ls->head != nullptr){
        ls->head->prev = nullptr;
    }else{
        ls->tail = nullptr;
    }
    delete temp;
    ls->n--;
}
```

DELETE FROM THE END OF DLL

Steps to delete a node from a specific position in a doubly linked list (DLL)

1. If the list is empty, do nothing and return
2. Save the last node (tail) in a temporary variable
3. Move the tail pointer to the previous node
4. If the new tail is not null, set its next pointer to null
5. Else, the list is empty, so set head to null
6. Delete the old tail node (free memory)
7. Decrease the count of nodes in the list by 1

```
void deleteEnd(List* ls){
    if(ls->n==0){
        return;
    }
    Element* tmp = ls->tail;
    ls->tail = tmp->prev;
    if(ls->tail != nullptr){
        ls->tail->next = nullptr;
    }else{
        ls->head = nullptr;
    }
    delete tmp;
    ls->n--;
}
```

DELETE FROM THE POSITION IN A DLL

Steps to delete a node from a specific position in a doubly linked list (DLL)

1. If the position is invalid, return
2. If position is 0, delete from beginning
3. If position is the last node, delete from the end
4. Otherwise
   - Traverse to the node before the one at the given position
   - Save the node to delete in a temporary variable
   - Update the previous and next pointers to skip the node
   - Delete the node
   - Decrease the node count

```
void deletePos(List* ls, int pos){
    if(pos<0|| pos>ls->n-1){
        return;
    }
    if(pos==0){
        deleteBeg(ls); return;
    }
    if(pos==ls->n-1){
        deleteEnd(ls); return;
    }

    Element* current = ls->head;
    for(int i=0;i<pos-1;i++){
        current = current->next;
    }
    Element* tmp = current->next;
    current->next = tmp->next;
    if(tmp->next != nullptr){
        tmp->next->prev = current;
    }
    delete tmp;
    ls->n--;
}
```

IMPLEMENTATION OF DOUBLY LINKED LIST IN C++

```
void deleteBeg(List* ls){
    if(ls->n==0) return;

    Element* temp = ls->head;
    ls->head = temp->next;
    if(ls->head != nullptr){
        ls->head->prev = nullptr;
    }else{
        ls->tail = nullptr;
    }
    delete temp;
    ls->n--;
}
```

```
void deleteEnd(List* ls){
    if(ls->n==0){
        return;
    }
    Element* tmp = ls->tail;
    ls->tail = tmp->prev;
    if(ls->tail != nullptr){
        ls->tail->next = nullptr;
    }else{
        ls->head = nullptr;
    }
    delete tmp;
    ls->n--;
}
```

```
void deletePos(List* ls, int pos){
    if(pos<0|| pos>ls->n-1){
        return;
    }
    if(pos==0){
        deleteBeg(ls); return;
    }
    if(pos==ls->n-1){
        deleteEnd(ls); return;
    }

    Element* current = ls->head;
    for(int i=0;i<pos-1;i++){
        current = current->next;
    }
    Element* tmp = current->next;
    current->next = tmp->next;
    if(tmp->next != nullptr){
        tmp->next->prev = current;
    }
    delete tmp;
    ls->n--;
}
```

```
int main(){
    List* list1 = createEmptyList();
    for(int i=0;i<10;i++){
        addBeg(list1,i);
    }
    displayBeg(list1);

    List* list2 = createEmptyList();
    for(int i=10; i<20;i++){
        addEnd(list2, i);
    }
    displayEnd(list2);

    addPos(list2, 99, 3);
    displayBeg(list2);

    deleteBeg(list2);
    displayBeg(list2);

    deleteEnd(list2);
    displayBeg(list2);

    deletePos(list2, 2);
    displayBeg(list2);
}
```

PRACTICE

1. Create a doubly linked list that can store integer numbers. Then
    a. Create two data structure (Element and List), and create an empty singly linked list
    b. Create function to add the element to the end of the list
    c. Create function to add the element to the beginning of the list
    d. Create function to display all numbers in the list
    e. Create function to delete the first element from the list
    f. Create function to delete the last element the list

PRACTICE

2. Create a doubly linked list that can store id, names of students.
    a. Create structure of student, element, list, and create an empty singly linked list.
    b. Create a function to add the student's info to the beginning of the list
    c. Create a function to add the student's info to end of the list
    d. Display number of students and all students in the list
    e. Create a function to search by student id in the list and display the student's name if found.
    f. Create a function to Update the name for the input student id

Q&A